

AT A GLANCE 주요 개발 경험

핵심 프로젝트



ShiftLog

26.03~26.06

#단서 수집 #스토리 어드벤처 #단서 수집

전체 UI, 단서 보드, 대화 시스템



텍사스 빙고

25.09~26.03

#산업협력 #전략보드게임 #PC/Mobile WebGL

족보 판정, 점수 규칙



텍사스 빙고 멀티

25.11~25.12

#온라인 보드게임 #Photon PUN2 #캐릭터 능력

턴 진행 상태 머신, 네트워크 동기화



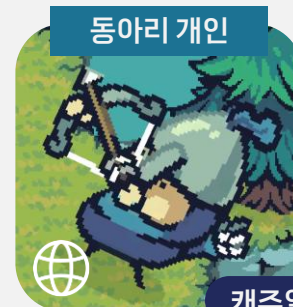
다양한 프로젝트



탄막 슈팅



전투 액션



캐주얼



디펜스

다양한 장르와 형태의 게임 프로젝트 진행
핵심 플레이와 시스템을 구현한 경험

ShiftLog

조사형 스토리 어드벤처게임

| 기간

26.03~26.06(4개월)

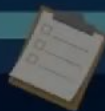
| 팀원

기획1, 아트4, 프로그래밍4

| 역할

프로그래밍 리드 및 UI/시스템 담당

복도



R



T



MY CONTRIBUTION 핵심 기여 사항

팝업 UI

Popup UI는 메인 화면 위에 열리며 공통생명주기를 사용하는 UI입니다

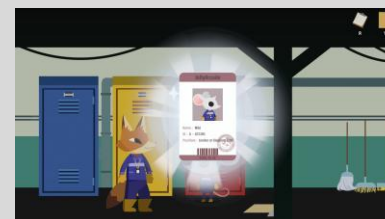
UIManager는 Popup별 관리 정책에 따라 활성 상태와 신규 Popup 요청을 처리합니다

설계 목표

청소 Popup이 열린 상태에서 증거/아이템을 획득할 경우
기존 팝업을 유지하면서 획득 알림 UI를 그 위에 표시해야 했습니다

획득 알림은 기존 Popup의 생성, 연출 등 생명주기를 공유하지만
기존 팝업UI와 동일한 추적 구조에는 맞지 않다고 판단했습니다.
이에 별도 속성을 추가하여 Manager의 추적 대상 여부를 구분했습니다

최종적으로 팝업UI는 배경 딤 처리 여부,
연출 중 예외 처리, 매니저 추적여부로 구성됩니다



<팝업UI 활성화 이미지>

MY CONTRIBUTION 핵심 기여 사항

바인더 UI

인벤토리, 증거함, 설정을 하나의 바인더 형태로 통합한 Popup UI입니다

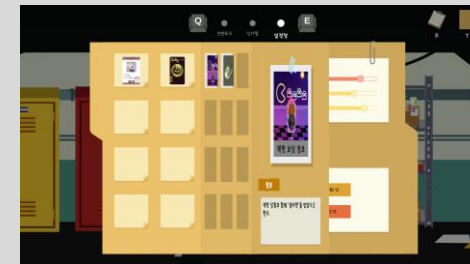
UI 등장 시 닫힌 상태로 나타나며 펼쳐지면서
페이지가 넘어가는 연출과 함께 선택한 페이지에 도달합니다

설계 목표

탭마다 서로 다른 페이지와 상호작용을 제공하면서 새로운 기능이 추가되어도
공통 페이지 전환 로직을 재사용할 수 있어야 했습니다

각 탭을 왼쪽, 오른쪽 페이지와 고유 상호작용을 가진 Section으로 분리했습니다
BinderUI는 Section의 순서와 공통 페이지 넘김만 관리하며
파생 Section이 각 기능의 Content와 상호작용을 정의합니다

새로운 기능은 파생 Section을 생성하고 순서에 등록하는 방식으로 추가할 수 있으며
기존 탭 전환과 페이지 넘김 로직은 그대로 재사용됩니다



목표 탭 도달까지 페이지 이동



중앙을 기준으로 요소가 변경



연출 완료 후 페이지의 입력 활성화

MY CONTRIBUTION 핵심 기여 사항

증거 보드 UI

수집한 증거를 질문지에 배치하고 사건의 연결 관계를 확인하는 조사 UI입니다

질문 마다 요구하는 증거의 개수와 방식이 달라 증거가 가로, 세로로 배치될 수 있고 배치된 증거의 다음 질문이 증거의 위치 옆에 생성됩니다

설계 목표

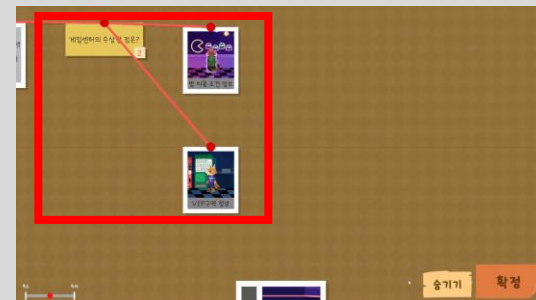
가변적으로 파생되는 질문, 증거 관계를 개별 예외 처리 없이 일관된 규칙으로 배치하고 연결해야 했습니다

첫 질문을 기준으로 우측·하단 방향으로 확장되는 Grid 구조를 적용하고 보드에 존재하는 증거, 질문지, 해설지를 공통 UI_BoardItem에서 파생시켰습니다

질문, 증거 조합이 달라져도 좌표 규칙에 따라 화면을 구성하고 공통 배치, 연결선 연출을 재사용할 수 있게 되었습니다



증거에 Hold 시 선택 연출



질문에 증거 드랍 시 순서대로 배치



올바른 증거 시 다음 질문으로 확장

MY CONTRIBUTION

핵심 기여 사항

팀 환경에 맞춘 협업 규칙 정립

프로그래머마다 달랐던 네이밍 방식과 코드 작성 형식을 통일하고 각자의 변경 내용을 팀에서 지속해서 확인할 수 있어야 했습니다

또한 새로운 협업 절차가 일회성 규칙으로 끝나지 않도록 팀원이 부담 없이 계속 이용할 수 있는 방식이어야 했습니다

운영 방식

별도의 프로젝트 관리 도구를 추가로 도입하기보다 팀이 기존부터 사용하던 디스코드에서 프로그래머 전용 채널을 만들어 협업 정보의 공유 공간으로 활용했습니다.

네이밍 컨벤션과 예시 코드를 공유하고, 작업 커밋 시 공유 채널에 변경 기능을 공유하여 서로에게 작업 현황을 공유할 수 있도록 했습니다. 실제 프로젝트는 Git으로 관리하고, Discord는 변경 사항을 빠르게 인지하기 위한 공유 수단으로 활용했습니다

팀원들이 익숙한 소통 환경에서 공통 규칙과 변경 내용을 확인하고, 다른 팀원의 작업 진행과 시스템 간 변경 영향을 지속해서 공유할 수 있도록 했습니다

● 인터페이스(Interface)

- I+ 형용사 / 역할
- "무엇을 할 수 있는가" 중심

✓ Good

```
IClickable
IPointable
IUsable
IBingoLineAbility
```

✗ Bad

```
IClick
IUse
IThing
```

1-2. git commit 시 # 커밋 에 공유

- 양식은 다음과 같이 작성

```
## <이니셜 (D, H, T, S)><날짜>. <작업 제목>
- <간단한 작업 내용>
```

Ex)

D001. 플레이어 움직임 수정

- 키 입력 정도에 따른 점프 높이 수정
- 관성 추가

2. 공용 씬/스크립트 수정 필요 시

- 미리 # 단독 에 수정 작업을 말하기

```
Test 씬 / 스크립트 작업 시작
- 인벤토리 UI 적용 및 테스트
- 예상 종료 시간: 오후 11시 (예상 가능 시에만)
```

- 이미 작업 중인 사람이 있다면 완료 대기 혹은 상의하기
- 작업이 끝난다면 커밋 후 작업 메시지에 ✓ 표시

- Project Setting/Sprite Setting과 같은 간단한 작업은 커밋 이전(작업 실행 후 바로) # 단독 에다가 내용 공유만

```
인벤토리UI 스프라이트 Border값 조정
```

2026-04-01 오전 2:19

D013. 바인더 UI 수정

- 증거함, 설정창 UI 배치
- 커버 인덱스 관리 수정

2026-04-01 오전 10:42

H017. Area.cs, Player.cs 수정

- Player 맵 이동 상호작용 추가
- 상호작용 바인드 키 UI 추가

H018. Prefab 수정

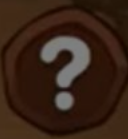
- Player 프리팹 수정
- NPC 프리팹 수정

(수정됨)

2026-04-01 오후 1:16

T017. BaseObject수정, EventRunner추가

- 강제 이벤트 추가
- BaseObject -> TopPosition 변수 수정



텍사스 빙고

보드게임 디지털화

| 기간

25.09~26.03(6개월)

| 팀원

기획2, 아트3, 프로그래밍2

| 역할

핵심 기능 구현 및 인게임 구현



완료

MY CONTRIBUTION 핵심 기여 사항

빙고판

텍사스 빙고는 5×5 빙고판에 플레이잉 카드를 배치하고
가로, 세로, 대각선으로 구성된 총 12개의 줄에서 5장 포커 족보를 만들어
각 줄 별로 최고의 족보를 만들어 점수를 획득하는 전략 보드게임입니다

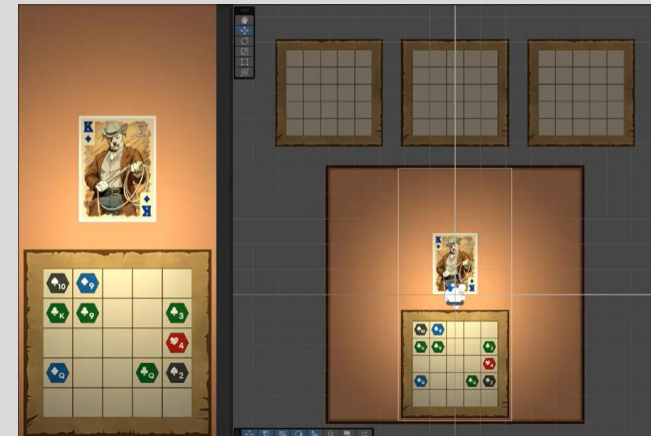
빙고판은 BingoBoard, BingoLine, BingoCell로 구성됩니다

설계 목표

하나의 Cell에 Card가 배치될 때 해당 Card가 포함된 Line을 판별하고
각 Line의 족보와 점수 상태를 갱신해야 했습니다

Cell 단위로 클릭을 감지하고 X, Y 좌표를 기준으로 가로와 세로 Line을 갱신하고,
대각선 조건을 충족하는 경우에만 해당 대각선 Line을 추가로 갱신하도록 구성했습니다

총 25개의 Cell, 12개의 Line과 1개의 Board로
텍사스빙고의 핵심 기능인 빙고판을 구현할 수 있었습니다



개발 초기 핵심 기능 프로토타입

MY CONTRIBUTION

핵심 기여 사항

족보 판별

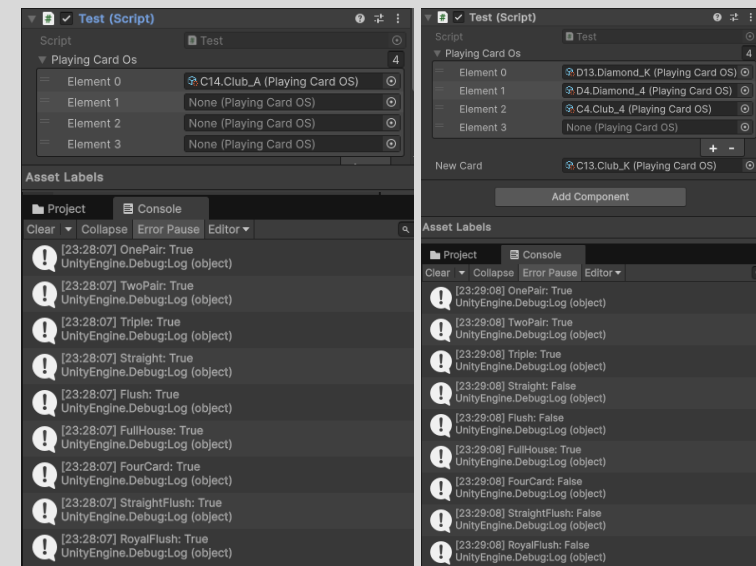
각 Line에 Card가 한 장씩 추가될 때마다
현재 달성 가능한 족보를 판정하는 시스템입니다

설계 목표

Line이 완성되기 전에도 족보 가능 상태를 유지하고
새 Card로 인해 달성할 수 없게 된 족보는 이후 판정에서 제외할 수 있도록 구성했습니다

기존 Card 구성과 추가된 Card를 비교하여 각 족보의 완성 가능성을 갱신했습니다
불가능 상태로 전환된 족보는 이후 Card가 추가되더라도
다시 판정하지 않고 해당 상태를 유지하도록 구성했습니다

5장이 완성되기 전에도 현재 가능한 족보 상태를 유지하고
불가능이 확정된 족보의 반복 판정을 생략할 수 있게 되었습니다



미완성 카드 구성에 대한 족보 가능성 출력

MY CONTRIBUTION

핵심 기여 사항

족보 분석과 참조 분리

점진적 족보 판정을 구현한 뒤, 기존 Card 구성에 새 Card 기반 족보 가능성 비교 시 비슷한 족보를 비교 시 반복해서 분석하는 중복 발생했습니다

스트레이트 족보 완성이 불가능하게 바뀌어도 스트레이트플러시 족보에 대해 한 번 더 분석하는 일이 발생했습니다

설계 목표

여러 족보가 공유하는 Card 분석을 한 번만 수행하고
각 Evaluator가 동일한 결과를 재사용하도록 책임을 분리하고자 했습니다

HandAnalysis에서 숫자·무늬와 Straight·Flush 가능성을 분석하여 Cache하고
각 족보 Evaluator가 해당 결과를 참조하도록 변경했습니다

중복되던 Card 분석을 공통화하고
각 Evaluator가 개별 족보 가능성 판정에 집중하도록 구조를 정리했습니다

```
public class HandAnalysis
{
    [파생 데이터]
    #region 상태 플래그
    참조 5개
    public bool CanBeStraight { get; private set; } //연속 가능 여부
    참조 5개
    public bool CanBeFlush { get; private set; } //동일 무늬 여부
    참조 8개
    public int CardCount => Cards.Count; //현재 패에 포함된 카드 수
    참조 4개
    public bool OnlyBroadway { get; private set; } //로얄 플러시 가능 여부
    참조 4개
    public bool OnlyAceLow { get; private set; } //백 스트레이트 여부
    #endregion
    참조 1개
    public HandAnalysis()
    참조 1개
    public void Update(PlayingCardType newCard)

    //동일 무늬 가능성이 있는지 확인
    참조 1개
    private bool CheckFlushPossibility()
    //연속 가능성이 있는 숫자인지 확인
    참조 1개
    private bool CheckStraightPossibility()
    /// 패에 같은 숫자가 ofKind개인 그룹의 개수 반환
    참조 12개
    public int GetKindCount(int ofKind, bool greaterThen = false)
    /// 패에 포함된 무늬 종류 개수 반환
    참조 1개
    public int GetSuitKindCount()
}
```

HandAnalysis가 갖고 있는 구성 멤버

텍사스 빙고 멀티

온라인 보드게임

| 기간

25.11~25.12(2개월)

| 팀원

기획2, 아트3, 프로그래밍2

| 역할

멀티플레이 변환 및 인게임 시스템 담당



MY CONTRIBUTION 핵심 기여 사항

멀티플레이 전환

기존 텍사스빙고와 차별점이 존재하며 네트워크 기반 게임으로 확장해야했습니다

메인 프로그래머로서 Multiplayer의 차별화 방향을 논의하고,
Network 환경에서 구현 가능한 Character 능력과
Turn 진행 방식을 검토하여 System 구조로 구체화했습니다

기존

로컬 기반 싱글 플레이
플레이어에게 동일한 기본 룰
실물 보드게임 경험 재현 중시
보드게임회사에서 제공한 리소스 기반
로컬 환경의 순차적 GameFlow

확장

PUN2 기반 네트워크 플레이
캐릭터 별 고유 능력 추가
디지털 게임 고유 경험 확장
Stylized 컨셉 리소스 재구성
Master Client 중심 Turn Flow



멀티플레이 최종 플레이 화면

MY CONTRIBUTION 핵심 기여 사항

턴 흐름

일반적인 네트워크 게임과 달리 각 Client에는 자신의 빙고판만 표시되며 다른 플레이어는 동일한 World의 GameObject가 아닌 Data로 존재합니다

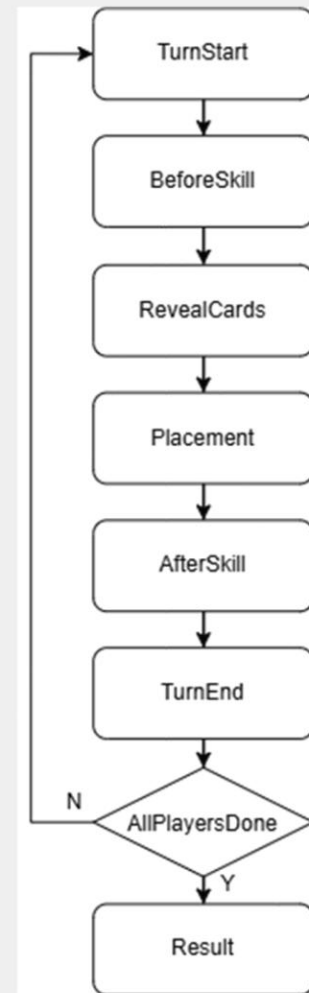
따라서 Object 상태보다 Player 행동의 순서와 완료 여부를 동기화하는 Turn 관리가 중요했습니다

설계 목표

Player마다 다른 Board에서 Card 배치와 Character 능력을 수행하더라도 모든 참가자가 행동을 완료한 뒤 다음 단계로 이동하도록 구성해야 했습니다

Turn을 다음과 같이 총 6개로 구분했습니다. 각 Client는 Phase별 행동을 완료한 뒤 Master Client에 보고하고, Master Client가 참가자의 완료 조건을 확인한 후 다음 Phase를 전파하도록 구성했습니다

Player별 행동 완료 시간이 달라도 Master Client가 공통 완료 조건을 기준으로 Phase를 전환하여 턴 흐름을 일관되게 관리할 수 있었습니다



게임 진행 흐름 진행도

MY CONTRIBUTION 핵심 기여 사항

캐릭터 고유 능력

각 캐릭터 별로 고유 능력을 하나씩 보유하고 있으며
발동 형태에 따라 패시브와 액티브로 구분됩니다

또한 발동 조건에 따라 카드 공개, 빙고 달성,
능력 시전, 카드 배치로 구분됩니다

설계 목표

각 `PlayerContext`를 초기화할 때 일괄적으로 캐릭터 능력을 Bind하여
발동 조건 달성 시 자동으로 발동하게 하도록 하였습니다

`CharacterSO`에서 파생된 각 캐릭터 별 능력 정의 후 `AbilityTriggerType`에 맞게
능력 실행을 올바른 호출부에 Bind합니다

다양한 캐릭터 고유 능력을 제공하지만 공통된 형태로
컨트롤할 수 있는 범위 내로 관리할 수 있게 하였습니다

```
#region Ability
[SerializeField] protected CharacterSkillInfo _skillInfo;
참조 4개
public CharacterSkillInfo SkillInfo => _skillInfo;
public AbilityTriggerType AbilityTrigger;
//현재 게임에 대한 정보는 없음

참조 3개
public virtual void OnBingoLineCompleted(PlayerContext owner, BingoLineRef lineRef) { }
참조 0개
public virtual void OnCardPlaced(PlayerContext owner, Vector2Int cell) { }
참조 6개
public virtual void OnTrigger(PlayerContext owner) { }
참조 0개
public virtual void OnPlaceCard(PlayerContext owner, Vector2Int cell) { }
#endregion
```

CharacterSO 능력 관련 멤버